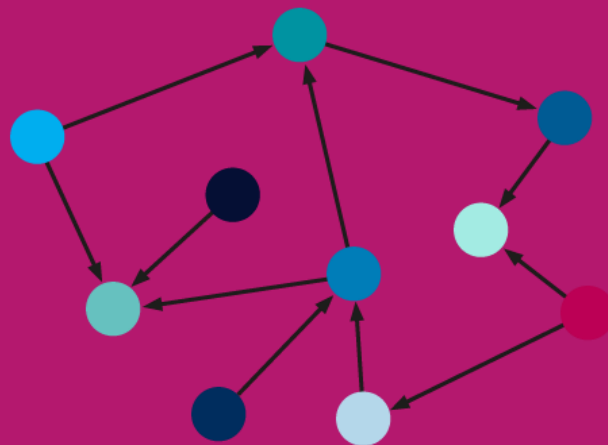




# Графові бази даних

Жишкович Андрій Володимирович

# Що таке графи?

Це спосіб представлення окремих об'єктів (сутностей) і зв'язків між ними.

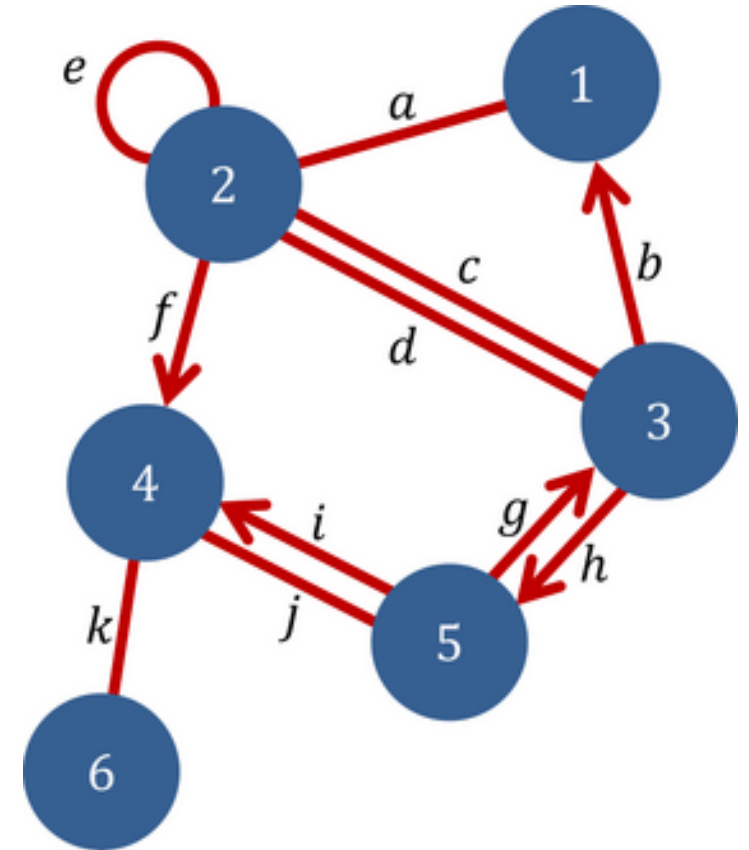
Графи дозволяють застосовувати різні алгоритми для аналізу цих зв'язків.

Графи можна знайти всюди, де об'єкти з'єднуються між собою.

Наприклад, у соціальних мережах ми маємо людей, які пов'язані один з одним через дружбу або знайомства.

В інфраструктурі це можуть бути лінії електропередач, які з'єднують трансформатори з електростанціями та будинками.

У залізничних мережах це можуть бути колії, точки перемикання та сигнали, які забезпечують роботу залізниці.



# Графові бази даних

**Графові бази даних** — це тип баз даних, які використовують графову структуру для зберігання, управління та запиту даних.

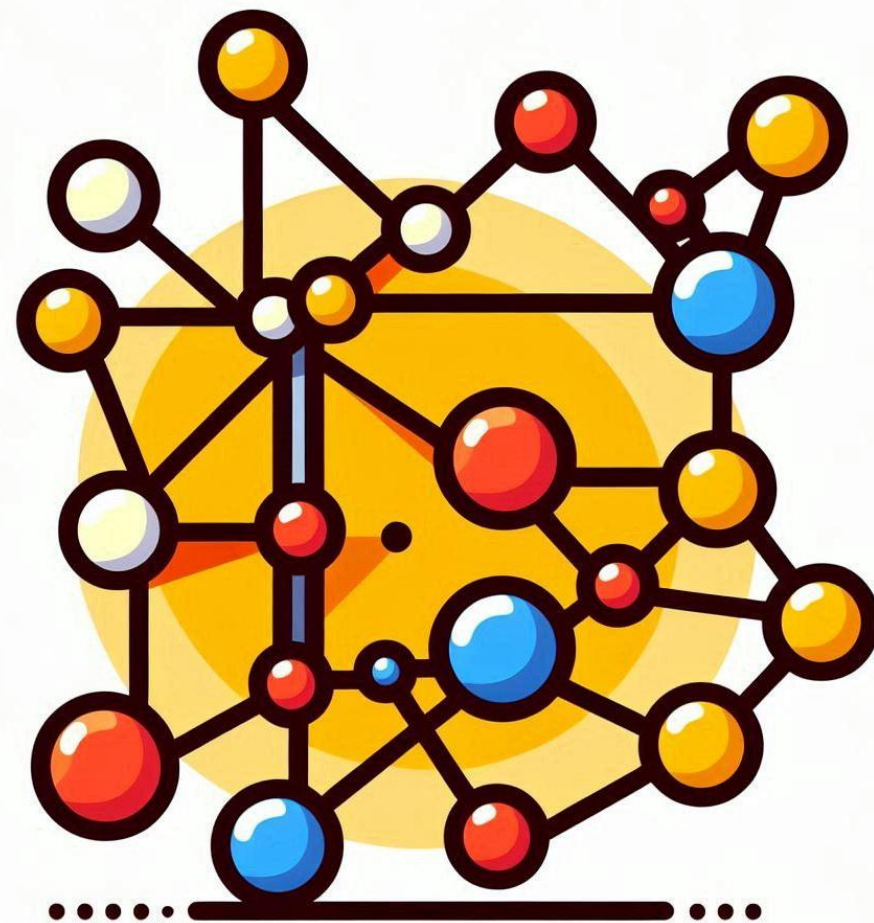
**Графова модель даних складається з:**

- **вузлів** (сутностей),
- **ребер** (зв'язків між сутностями)
- **властивостей** (додаткова інформація про вузли і ребра).

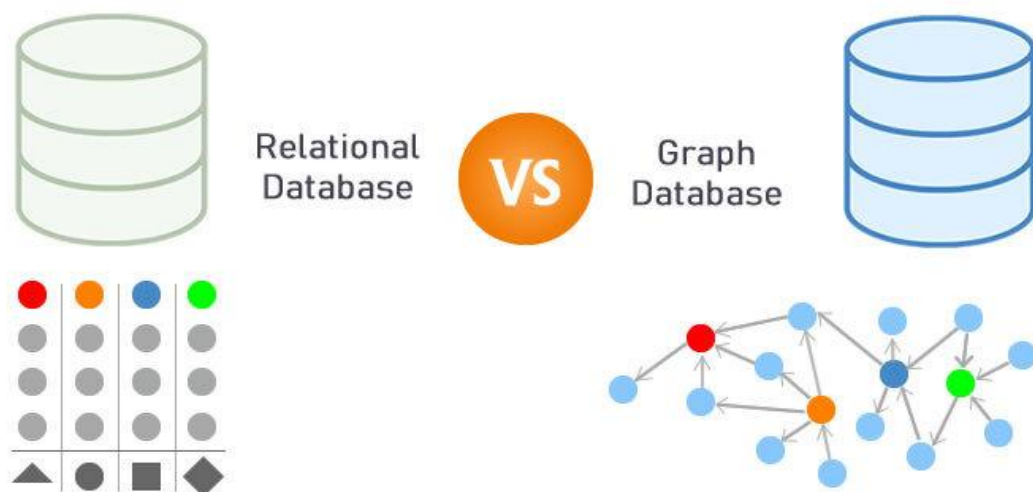
Графові бази даних часто мають спеціалізовану мову запитів для полегшення обходу зв'язків.

Вони є потужним інструментом для зберігання та обробки даних, які мають складні взаємозв'язки.

Якщо ви стикаєтеся з обмеженнями традиційних баз даних, особливо коли мова йде про складні взаємозв'язки даних, графові бази даних можуть стати саме тим рішенням, яке вам потрібно.



# Різниця між графовою БД та реляційною БД



Графові бази даних не замінюють реляційні бази даних. На даний момент реляційні бази даних є галузевим стандартом. Найважливіший аспект — знати, що може запропонувати кожен тип бази даних.

Реляційні бази даних забезпечують структурований підхід до даних, тоді як графові бази даних є гнучкими та зосереджені на швидкому аналізі зв'язків між даними.

І графові, і реляційні бази даних мають свою сферу застосування.

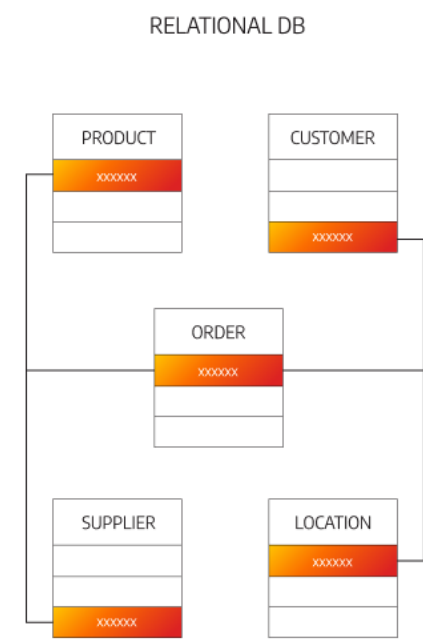
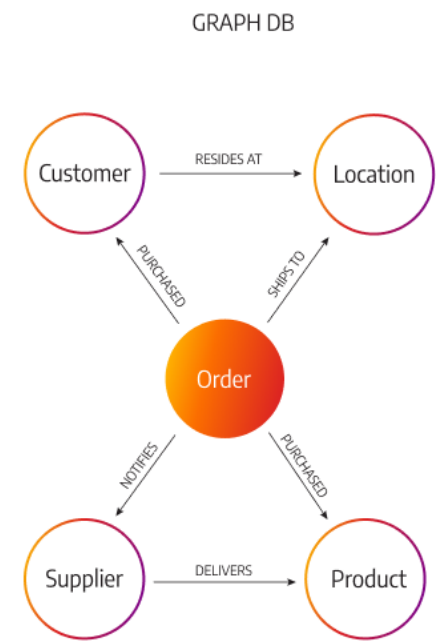
Потужність графових баз даних проявляється коли є багато даних зі складними зв'язками.

Реляційні бази даних, такі як MySQL або PostgreSQL, вимагають ретельного планування під час створення моделей баз даних, тоді як графи мають набагато більш природний і плавний підхід до даних.

# Різниця між графовою БД та реляційною БД

Критичні відмінності між графовими та реляційними базами даних:

| Тип                  | Графова БД                               | Реляційна БД                                                                      |
|----------------------|------------------------------------------|-----------------------------------------------------------------------------------|
| Формат               | Вузли та ребра з властивостями           | Таблиці з рядками і стовпцями                                                     |
| Зв'язки              | Представлені з'єднанням вершин з ребрами | Кожному значенню ключа повинна відповідати достатня інформація про об'єкт таблиці |
| Гнучкість            | Гнучка                                   | Жорстка                                                                           |
| Складність запитів   | Швидкі запити та швидка реакція          | Потрібні складні з'єднання                                                        |
| Випадки використання | Системи з складними зв'язками            | Системи, орієнтовані на транзакції, з більш простими зв'язками                    |



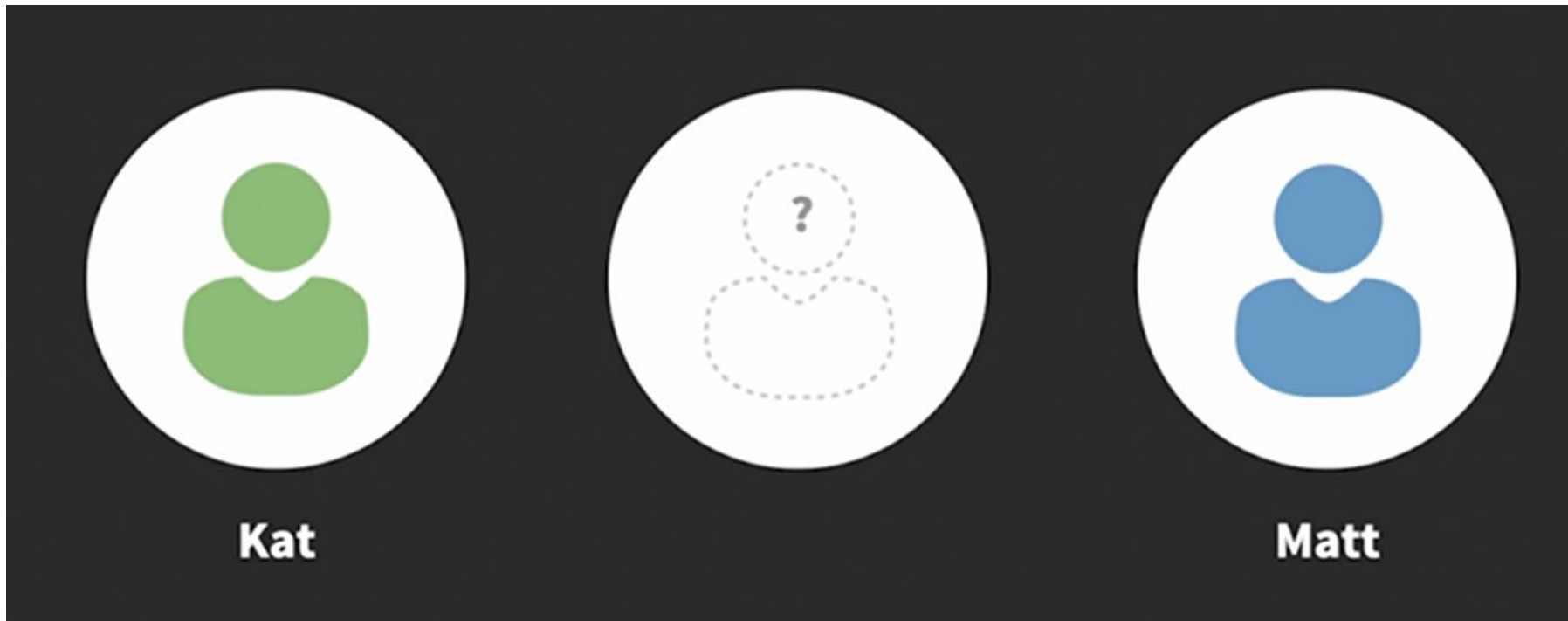
**Чому графові бази даних?**

# Чому графові бази даних?

Багато проблем з даними є проблемами взаємопов'язаних даних.

Розглянемо простий приклад: Kat, студентка графічного дизайну, хоче зв'язатися з Matt, але не знайома з ним.

Чи є спільний знайомий, який міг би їх познакомити?



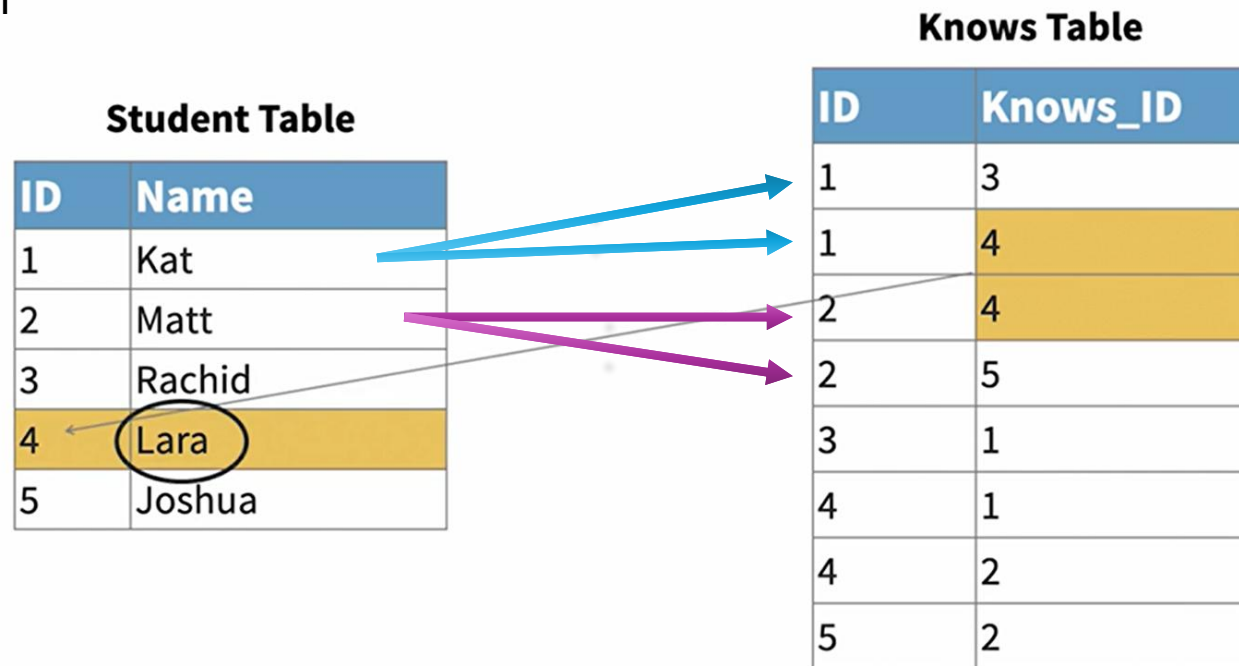
# Чому графові бази даних?

## Реляційна база даних

У реляційній базі даних ми нормалізуємо дані, видаляючи дублікати.

Для пошуку спільного знайомого нам потрібно виконати "join query", щоб об'єднати дані з таблиць студентів і знайомств.

Це працює для невеликих наборів даних, але стає складним для великих таблиць або у випадку відсутності спільних знайомих другого рівня.



# Чому графові бази даних?

## Документо-орієнтована база даних

У документній базі даних кожен об'єкт має масив друзів з їхніми ідентифікаторами (наприклад, MongoDB).

Ми отримуємо масиви друзів для Kat і Matt та шукаємо збіги.

Це також працює для невеликих наборів даних, але стає складним, коли ми виходимо за межі друзів другого рівня (зв'язків) чи у випадку видалення об'єктів.

```
{  
  "_id":1,  
  "name":"Kat"  
  friends: [3,4]  
}
```

```
{  
  "_id":2,  
  "name":"Matt"  
  friends: [4,5]  
}
```

```
{  
  "_id":3,  
  "name":"Rachid"  
  friends: [1]  
}
```

```
{  
  "_id":4,  
  "name":"Lara"  
  friends: [1,2]  
}
```

```
{  
  "_id":5,  
  "name":"Joshua"  
  friends: [2]  
}
```

# Чому графові бази даних?

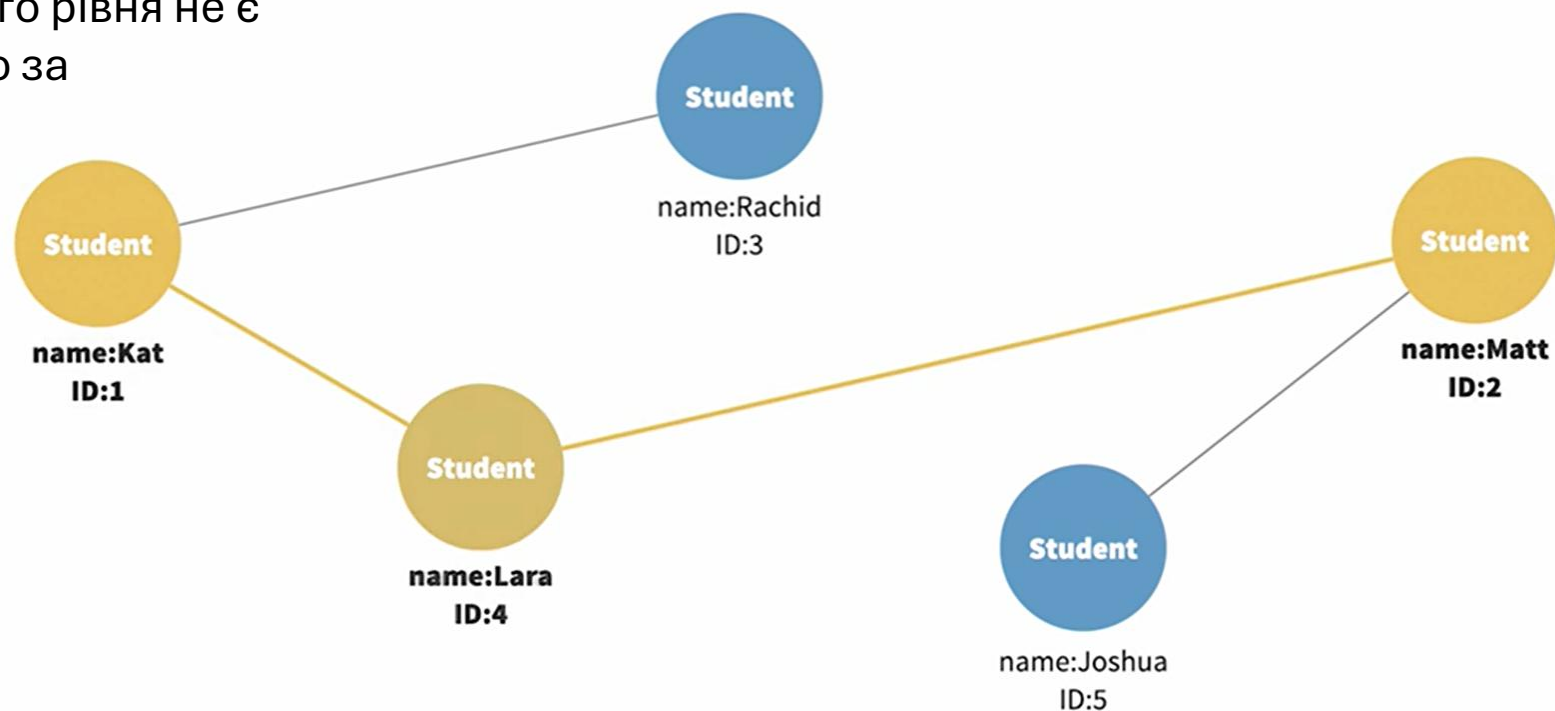
## Графова база даних

Графові бази даних створюють фізичні зв'язки між сутностями.

Якщо зв'язок існує, ми його швидко знайдемо.

Наприклад, ми швидко знайдемо, що Лара є спільним знайомим для Kat і Mett.

Пошук зв'язків третього, четвертого та n-го рівня не є проблемою, оскільки ми просто слідуємо за взаємозв'язками.



# **Хороші та погані випадки використання графових баз даних**

# Хороші та погані випадки використання графових баз даних

## Хороші випадки використання

Графові бази даних ідеально підходять для задач, де важливі взаємозв'язки між даними.

### Наприклад:

#### Соціальні мережі:

Визначення, як люди пов'язані один з одним.

#### Залежності систем:

Визначення, які системи постраждали від збою.

#### Виявлення шахрайства:

Аналіз взаємозв'язків між транзакціями для виявлення підозрілих дій.

#### Планування маршрутів:

Оптимізація логістики на основі дорожніх умов.

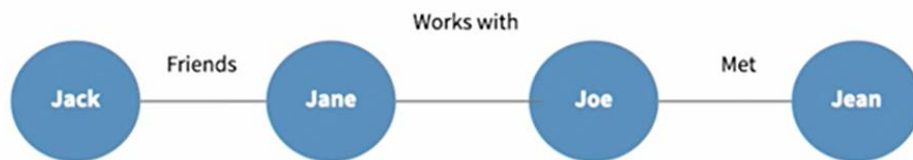
#### Рекомендаційні системи:

Аналіз попередніх виборів студентів для рекомендації курсів.

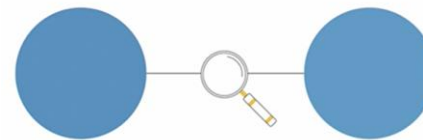
### Як пов'язані сутності?



### Дослідження довгих ланцюгів зв'язків



### Розуміння взаємовідносин між сутностями



# Хороші та погані випадки використання графових баз даних

## Погані випадки використання

Графові бази даних не підходять для задач, де взаємозв'язки не є ключовими.

### Наприклад:

#### Аналіз великих списків даних:

Обчислення середніх показників продажів за певний період.

#### Зберігання бінарних даних:

Зберігання зображень та інших файлів.



Дослідження списків даних

Регулярне отримання бінарних даних

Відсутність перегляду будь-яких пов'язаних даних

# Хороші та погані випадки використання графових баз даних

## Приклади з життя на кампусі

### Аналіз зборів за курс:

Аналіз квартальних зборів за курс "Математика 101" не потребує графової бази даних.

### Зберігання зображень:

Зберігання зображень для нового вебсайту краще виконувати в іншій базі даних.

### Планування маршруту доставки:

Оптимізація логістики для доставки екзаменаційних робіт — хороший випадок для графової бази даних.

### Система рекомендацій курсів:

Рекомендація курсів на основі попередніх виборів студентів — ідеальний випадок для графової бази даних.



## Висновок

Графові бази даних чудово підходять для задач, де важливі взаємозв'язки між даними. Для інших задач краще використовувати інші технології зберігання даних.

# Типи графових структур для даних

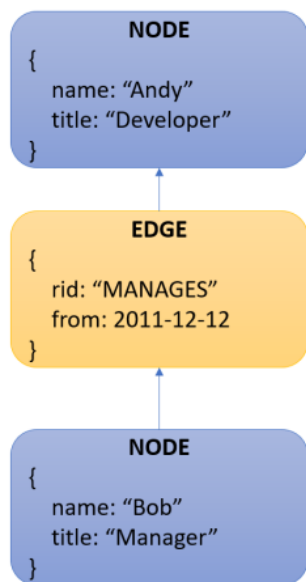
# Типи графових структур для даних

У графових базах даних використовуються різні типи структур для представлення взаємопов'язаних даних.

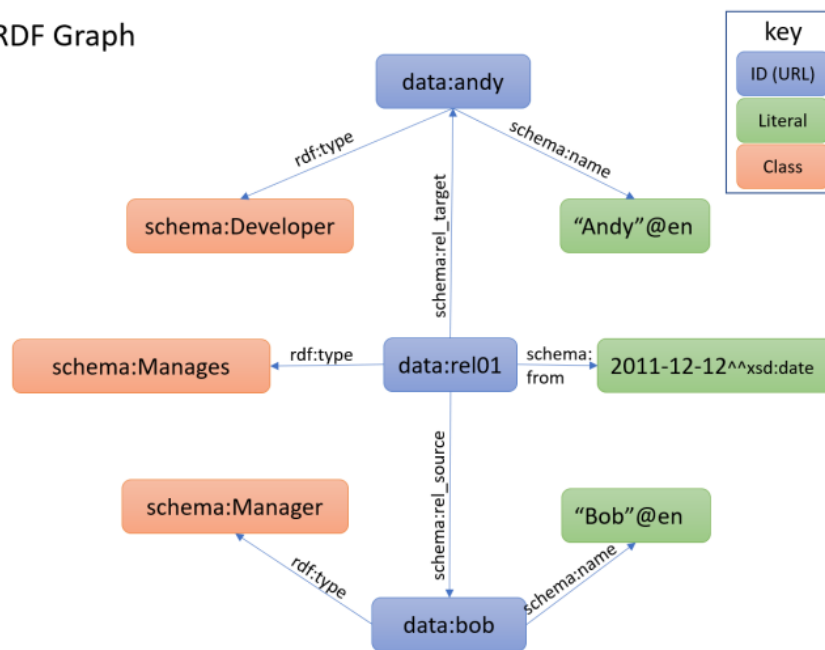
Найбільш поширеними є:

**Resource Description Framework (RDF)** та **Property Graph** або **Labeled Property Graph (LPG)**

Labelled Property Graph



RDF Graph



## Resource Description Framework (RDF)

— це стандарт для опису ресурсів у веб-просторі, який використовує концепцію трійок: суб'єкт-предикат-об'єкт.

## Property Graph або Labeled Property Graph (LPG)

— це структура даних, яка представляє дані у вигляді графа з вузлами (nodes) і ребрами (edges), де кожен вузол і ребро можуть мати властивості (properties).

# Resource Description Framework (RDF)

**Resource Description Framework (RDF)** — це стандарт для опису ресурсів у веб-просторі, який використовує концепцію трійок: суб'єкт-предикат-об'єкт.

**Суб'єкт (Subject):** Об'єкт, про який йде мова.

**Предикат (Predicate):** Взаємозв'язок або властивість суб'єкта.

**Об'єкт (Object):** Значення або інший об'єкт, пов'язаний із суб'єктом через предикат.

## Приклад:

Суб'єкт: "ідентифікатор один"

Предикат: "ім'я"

Об'єкт: "Kat"

Суб'єкт: "Студент1"

Предикат: "має ім'я"

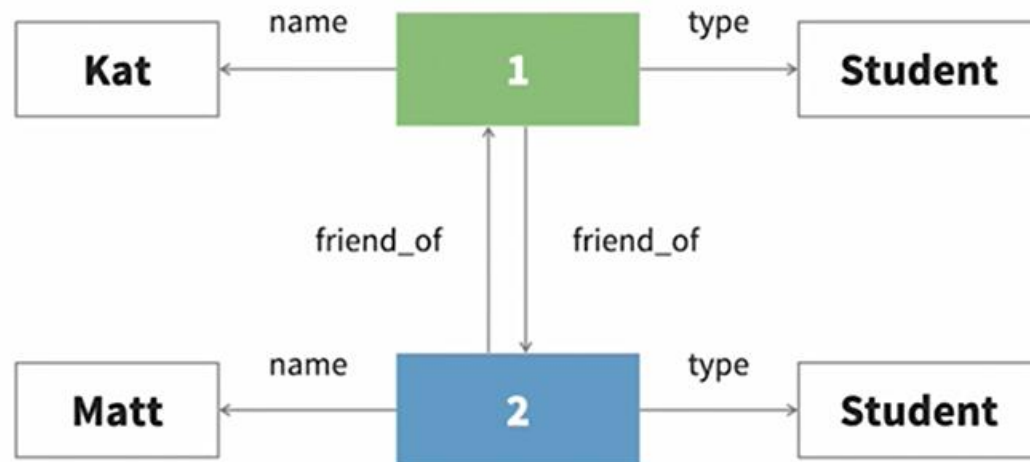
Об'єкт: "Kat"

Суб'єкт: "ідентифікатор один"

Предикат: "тип"

Об'єкт: "студент"

RDF добре підходить для семантичного моделювання та обміну даними між системами. Він є стандартом консорціуму World Wide Web (W3C) для обміну даними.



# Property Graph

**Property Graph або Labeled Property Graph (LPG)** — це структура даних, яка представляє дані у вигляді графа з вузлами (nodes) і ребрами (edges), де кожен вузол і ребро можуть мати властивості (properties).

**Вузли (Nodes):** Основні об'єкти або сутності.

**Ребра (Edges):** Взаємозв'язки між вузлами.

**Властивості (Properties):** Додаткові дані, які можуть бути прикріплені до вузлів і ребер.

**Мітки (Labels):** Категорії або типи вузлів.

## Приклад:

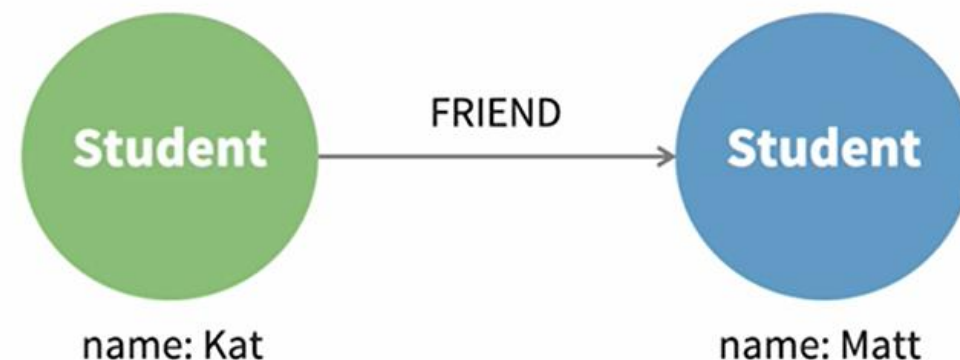
Вузли: "Kat" і "Matt"

Мітки: "Студент"

Ребро: "друг"

Властивості: "ім'я" з відповідними значеннями "Kat" і "Matt"

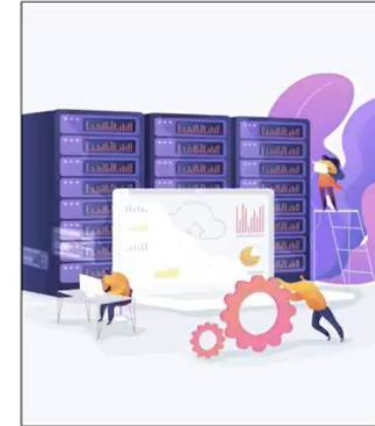
Property Graph добре підходить для транзакційних систем (наприклад, обробка покупок) та аналітичних систем (наприклад, створення звітів або розслідування минулих дій).



# Типові випадки використання

# Типові випадки використання графових баз даних

## ТОП-3



1

### Social Networks

Represent entities and their relationships

#### Соціальні мережі

Відстеження зв'язків між користувачами, друзями, підписниками тощо.

2

### Fraud Detection

Once again, NoSQL databases, in general, rule here

#### Шахрайство

Виявлення підозрілих шаблонів у фінансових транзакціях.

3

### Recommendations

Real-time insights connecting customer needs and products

#### Рекомендаційні системи

Виявлення зв'язків між продуктами та користувачами для надання персоналізованих рекомендацій.

# Які проблеми можуть вирішувати графові бази даних?

## Складні, взаємопов'язані дані

Графові бази даних легко справляються зі складними взаємозв'язками, де традиційні бази даних мають труднощі. Вони ідеальні для соціальних мереж, рекомендаційних систем та організаційних ієрархій.

## Виявлення шахрайства та оцінка ризиків

Графові бази даних швидко ідентифікують підозрілі шаблони та взаємозв'язки, що допомагає банкам та страховим компаніям виявляти шахрайство та оцінювати ризики.

## Рекомендації в реальному часі

Аналізуючи поведінку користувачів та взаємозв'язки між продуктами, графові бази даних надають персоналізовані рекомендації, що корисно для платформ електронної комерції та стрімінгових сервісів.

## Знання графів та онтології

Графові бази даних ефективно організовують та запитують складну інформацію, що робить їх корисними для досліджень, управління контентом та семантичного пошуку.

## Мережеві та ІТ-операції

Графові бази даних допомагають моделювати та аналізувати взаємозв'язки між компонентами мережі, що сприяє оптимізації продуктивності та виявленню проблем.

## Аналіз соціальних мереж

Графові бази даних дозволяють ефективно візуалізувати та аналізувати взаємозв'язки в соціальних мережах, що корисно для маркетингу та управління спільнотами.

# Використання графових баз даних у сфері охорони здоров'я

## Аналіз шляху пацієнта

Графові бази даних моделюють взаємодії пацієнтів у системі охорони здоров'я, допомагаючи виявляти прогалини в наданні медичної допомоги.

## Виявлення шахрайства

Графові бази даних аналізують взаємозв'язки між пацієнтами, постачальниками послуг та заявками, виявляючи підозрілі патерни.



# Використання графових баз даних у фінансовій сфері

## Виявлення шахрайства

Графові бази даних виявляють підозрілі шаблони транзакцій та зв'язки між сутностями, що допомагає виявляти фінансові злочини.

## Замовник 360

Графові бази даних інтегрують дані з різних джерел, надаючи комплексний огляд кожного клієнта для персоналізації пропозицій та покращення обслуговування.

## Оцінка ризиків

Графові бази даних аналізують взаємозв'язки між позичальниками, кредиторами та фінансовими транзакціями, покращуючи оцінку кредитних ризиків.



# Як Netflix використовує графові бази даних

## Представлення користувачів, фільмів та взаємодій

Графова база даних Netflix моделює користувачів, фільми та їх взаємодії, що дозволяє аналізувати поведінку користувачів.

## Персоналізовані рекомендації

Рекомендаційна система Netflix використовує графові бази даних для надання персоналізованих рекомендацій на основі взаємозв'язків між користувачами та фільмами.

## Масштабування та оновлення в реальному часі

Графова база даних Netflix ефективно обробляє великі обсяги даних та оновлення в реальному часі, забезпечуючи актуальність рекомендацій.



# Чи варті графові бази даних?

Графові бази даних пропонують значні переваги для складних, взаємопов'язаних даних. Вони ідеальні для соціальних мереж, рекомендаційних систем та виявлення шахрайства, забезпечуючи швидкі запити та інсайти в реальному часі.

## Переваги:

### Швидкі запити:

Швидкий перехід між взаємозв'язками вузлів.

### Реальні інсайти:

Ідеально підходять для додатків, що потребують аналізу в реальному часі.

## Виклики:

### Навчання:

Потребують вивчення нових концепцій та інструментів.

### Адаптація:

Вимагають початкового навчання та адаптації.

## Використання:

### Доповнення:

Графові бази даних доповнюють, а не замінюють інші бази даних.

### Специфічні сценарії:

Найкраще підходять для складних, взаємопов'язаних даних, але не для простих, плоских даних.

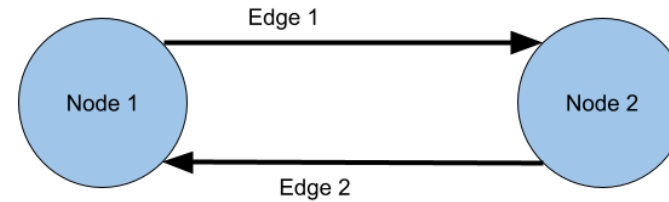
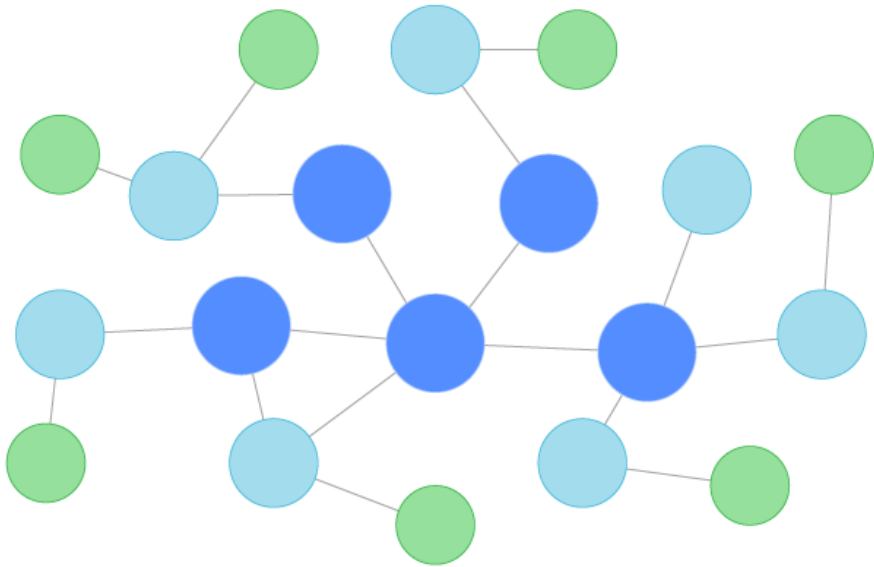
Графові бази даних можуть забезпечити рівень продуктивності та інсайтів, який інші бази даних не можуть зрівняти, особливо для складних взаємозв'язків.

# Основні концепції

# Основні концепції

Одна добра риса графів полягає в тому, що навіть якщо вони представляють абстрактне математичне визначення, ми можемо візуально відобразити їх за допомогою найпростіших малюнків кіл і ліній.

Ми можемо розглядати вузли як фундаментальні сутності графів, а ребра представляють взаємозв'язки між вузлами.



**Вершини (Nodes):** Основні елементи, які представляють сутності, такі як люди, місця, продукти тощо.

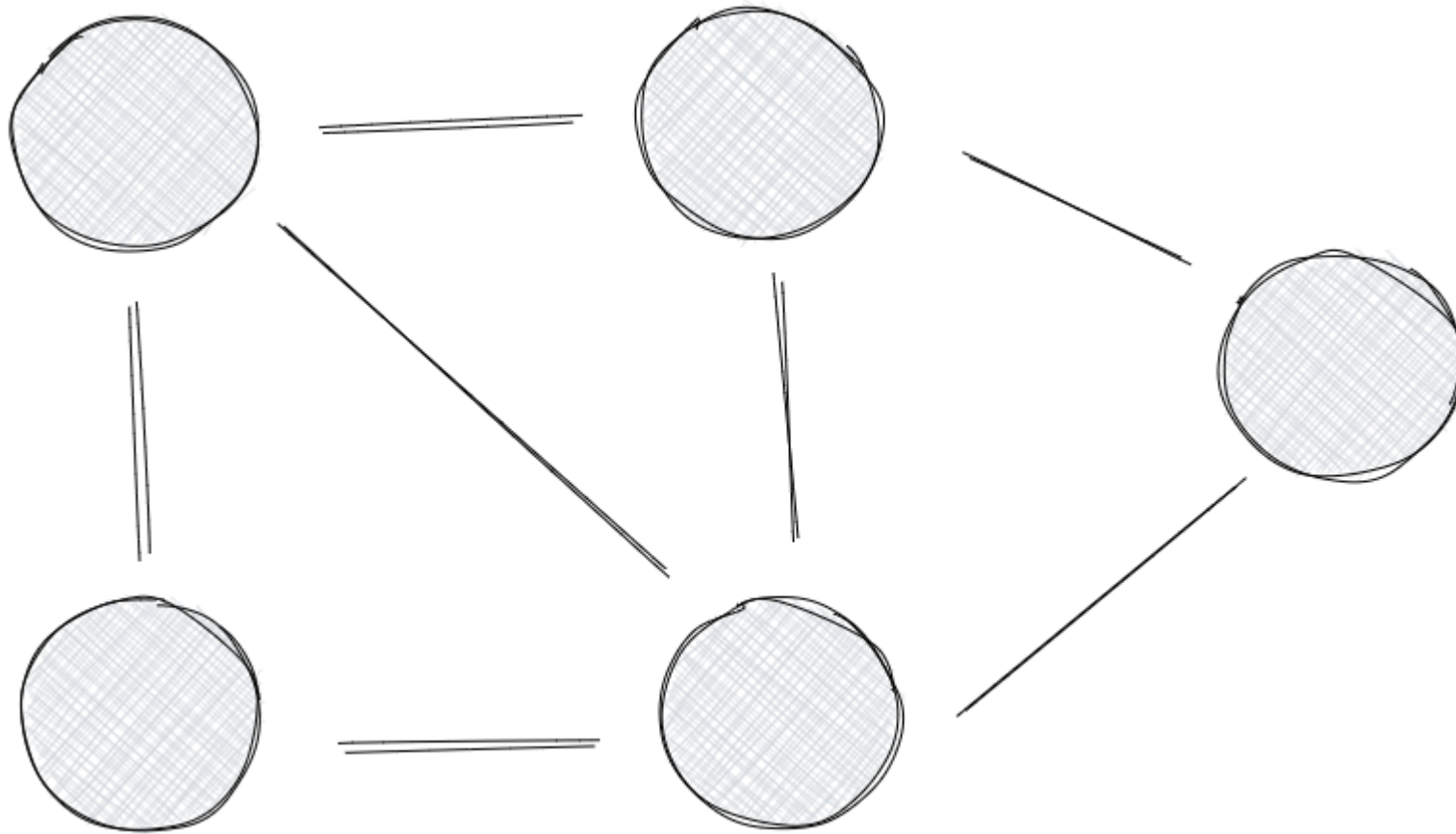
**Ребра (Edges):** Зв'язки між вершинами, які показують відносини між ними. Ребра можуть мати напрямок (орієнтовані графи) або бути без напрямку (неорієнтовані графи).

**Властивості (Properties):** Додаткові дані, які можуть бути прикріплені до вершин або ребер, наприклад, ім'я, вік, дата створення тощо.

У графах об'єкти представлені вузлами, а способи, якими ці об'єкти з'єднані між собою - взаємозв'язками.

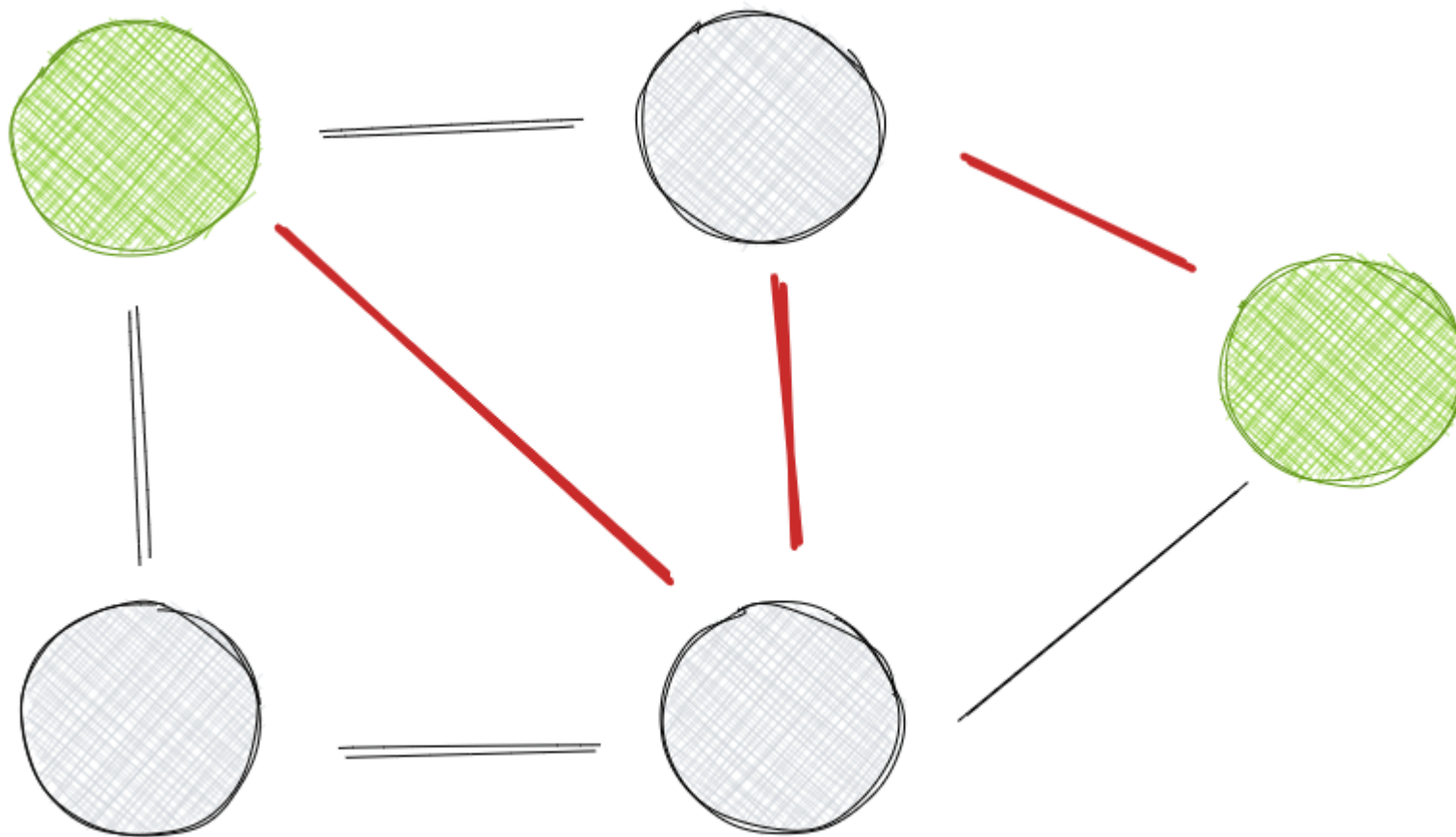
# Основні концепції – Вузли, Ребра та Шляхи

Ось намальоване від руки зображення графа з 5 вузлами і 7 ребрами, яке ми будемо використовувати для розуміння деяких базових визначень:



# Основні концепції – Вузли, Ребра та Шляхи

**Шлях** веде від **вузла а** до **вузла б** через ребра графа. **Довжина шляху** — це кількість ребер у ньому. Виділений (червоний) шлях між двома зеленими вузлами має довжину 3:

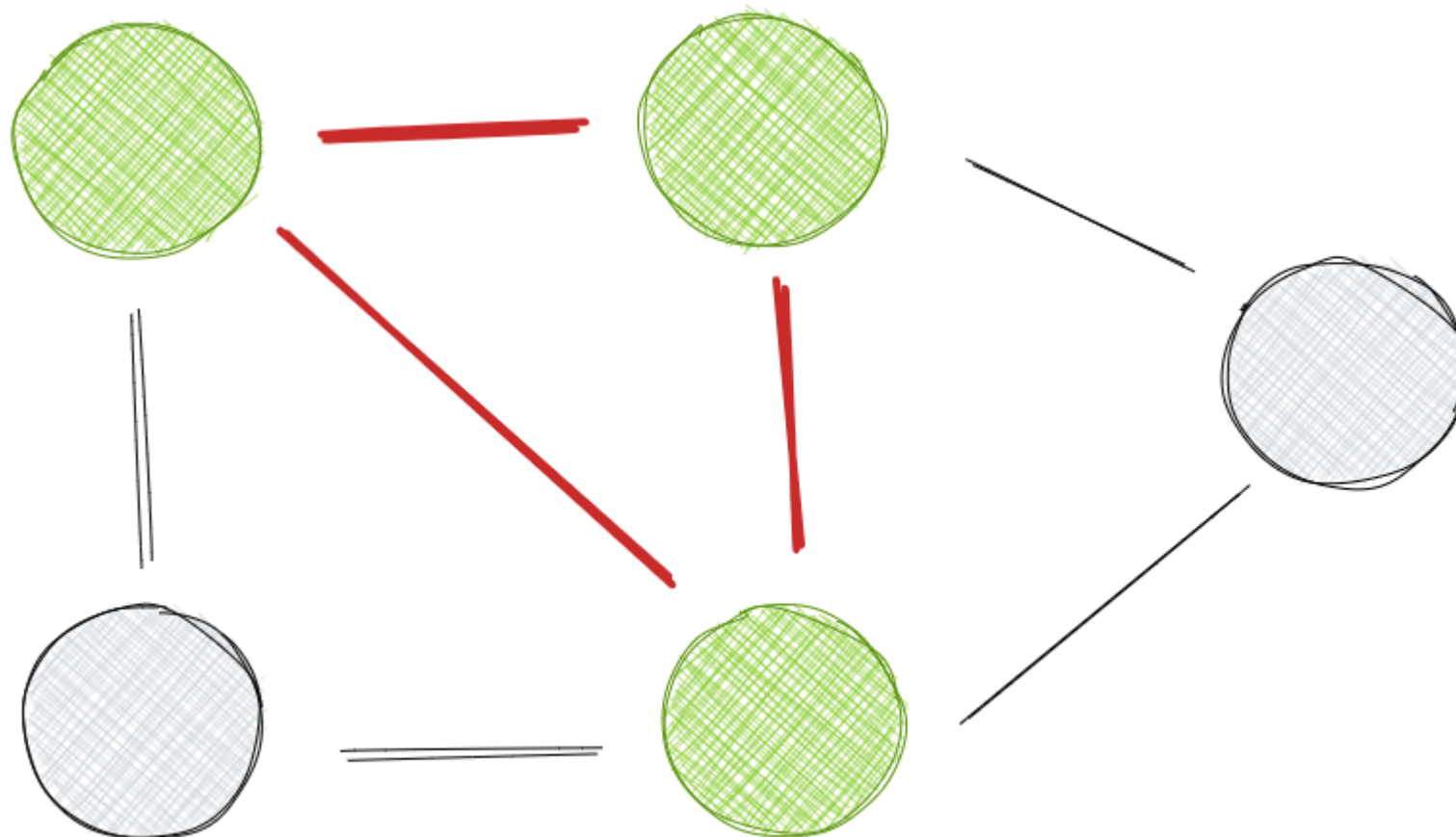


*Примітка: Виділений шлях — це лише один з можливих шляхів між двома зеленими вузлами. Спробуйте визначити всі можливі шляхи між ними.*

# Основні концепції – Вузли, Ребра та Шляхи

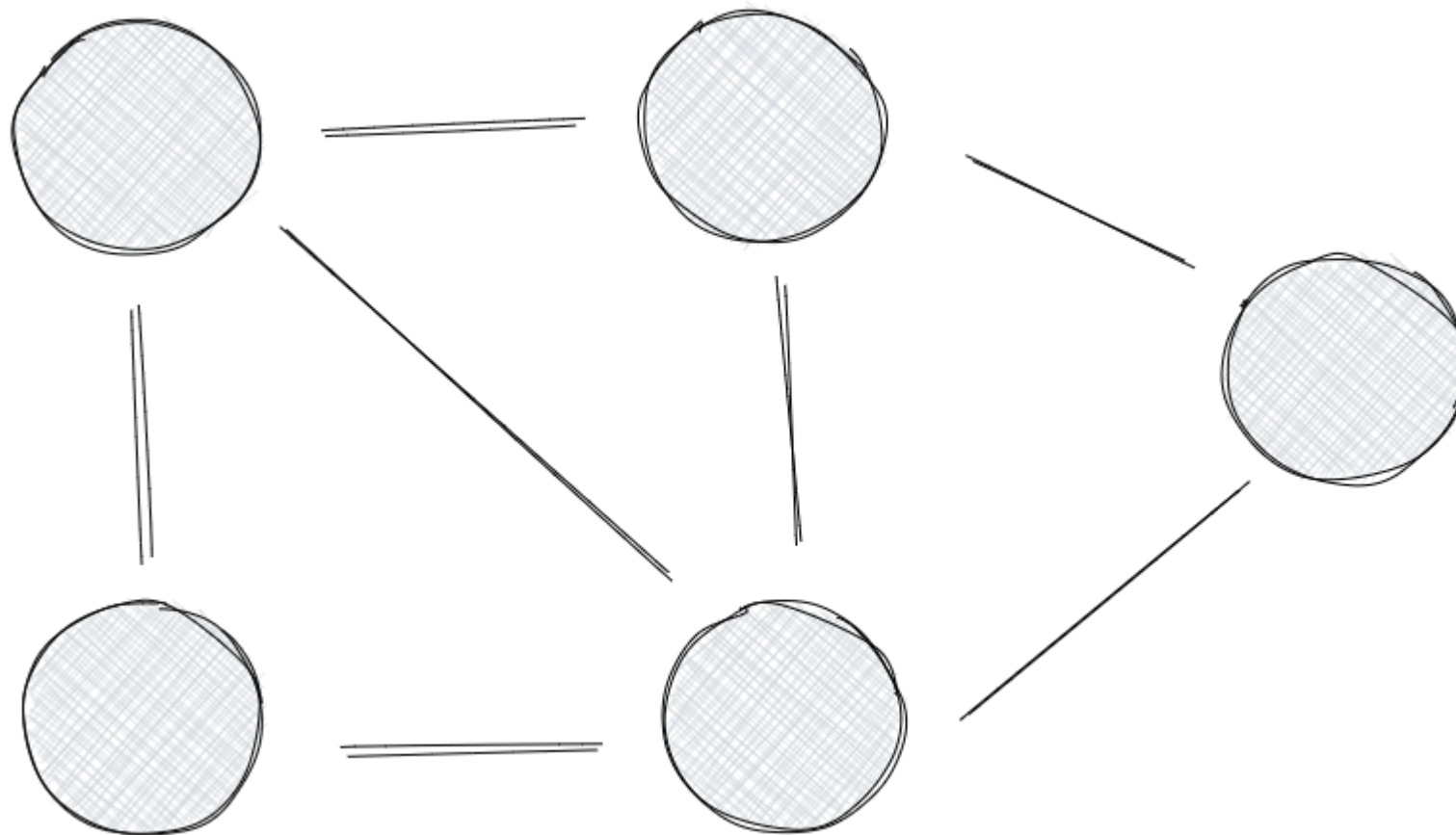
Шлях є **циклом**, якщо перший і останній вузол однакові. З іншого боку, шлях називається **простим**, якщо кожен вузол з'являється в ньому не більше одного разу.

Виділений шлях на попередньому зображенні відповідає простому шляху, тоді як на наступному зображенні він вважається циклом:



# Основні концепції – Зв'язність

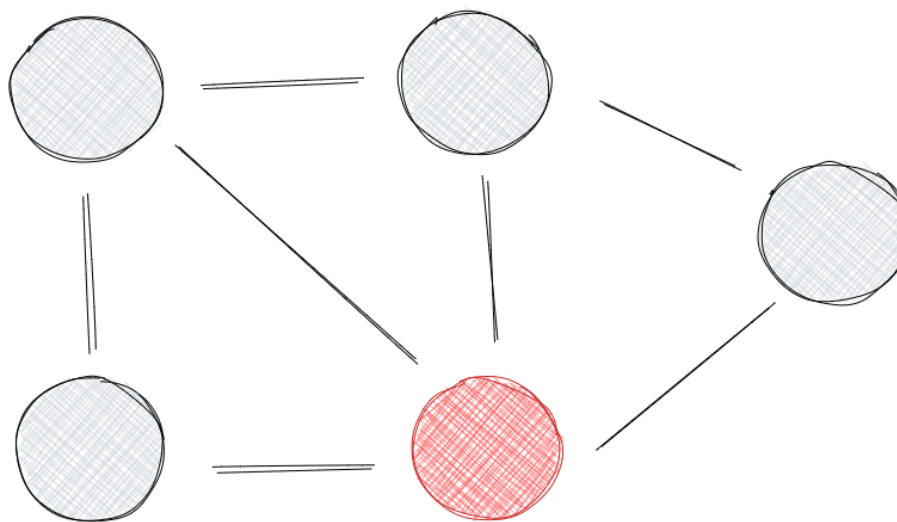
Граф є **зв'язним**, якщо між будь-якими двома вузлами існує шлях. Наприклад, наш початковий граф є зв'язним:



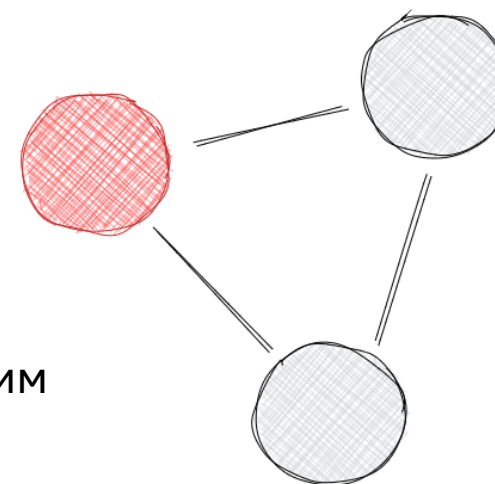
*Спробуйте самі: візьміть будь-яку пару вузлів у попередньому графі та перевірте, що між ними дійсно існує шлях.*

# Основні концепції – Зв'язність

З іншого боку, наступний граф **не є зв'язним**, оскільки між виділеними вузлами немає шляху:



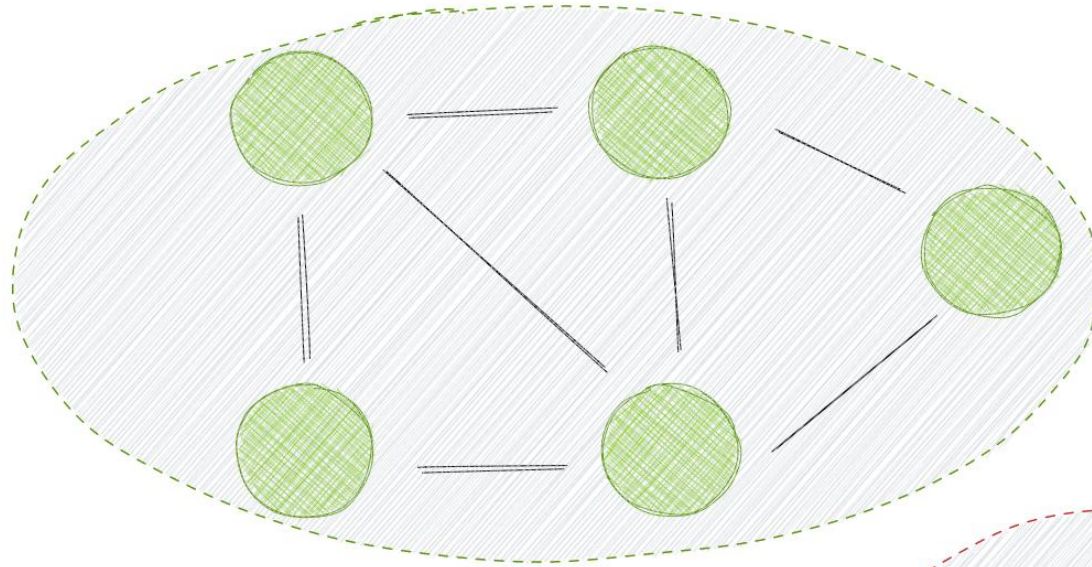
Насправді, немає шляху між жодним вузлом, розташованим у правому нижньому куті, і жодним вузлом, розташованим у лівому верхньому куті.



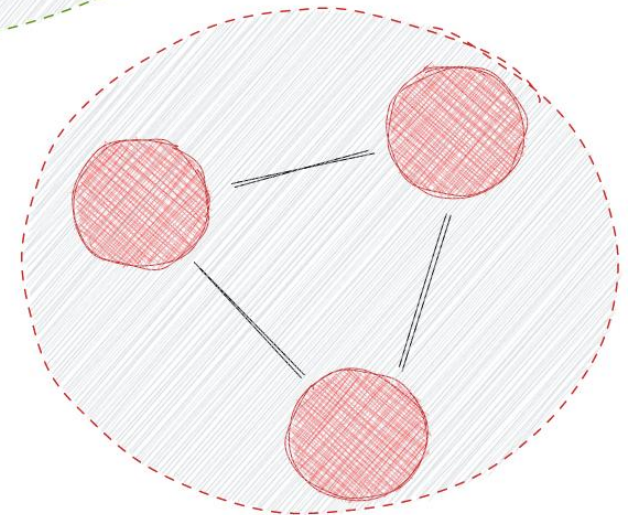
# Основні концепції – Зв'язність

Набори зв'язаних вузлів у графі називаються **зв'язаними компонентами**.

Ось зв'язані компоненти (вузли одного кольору) попереднього графа:



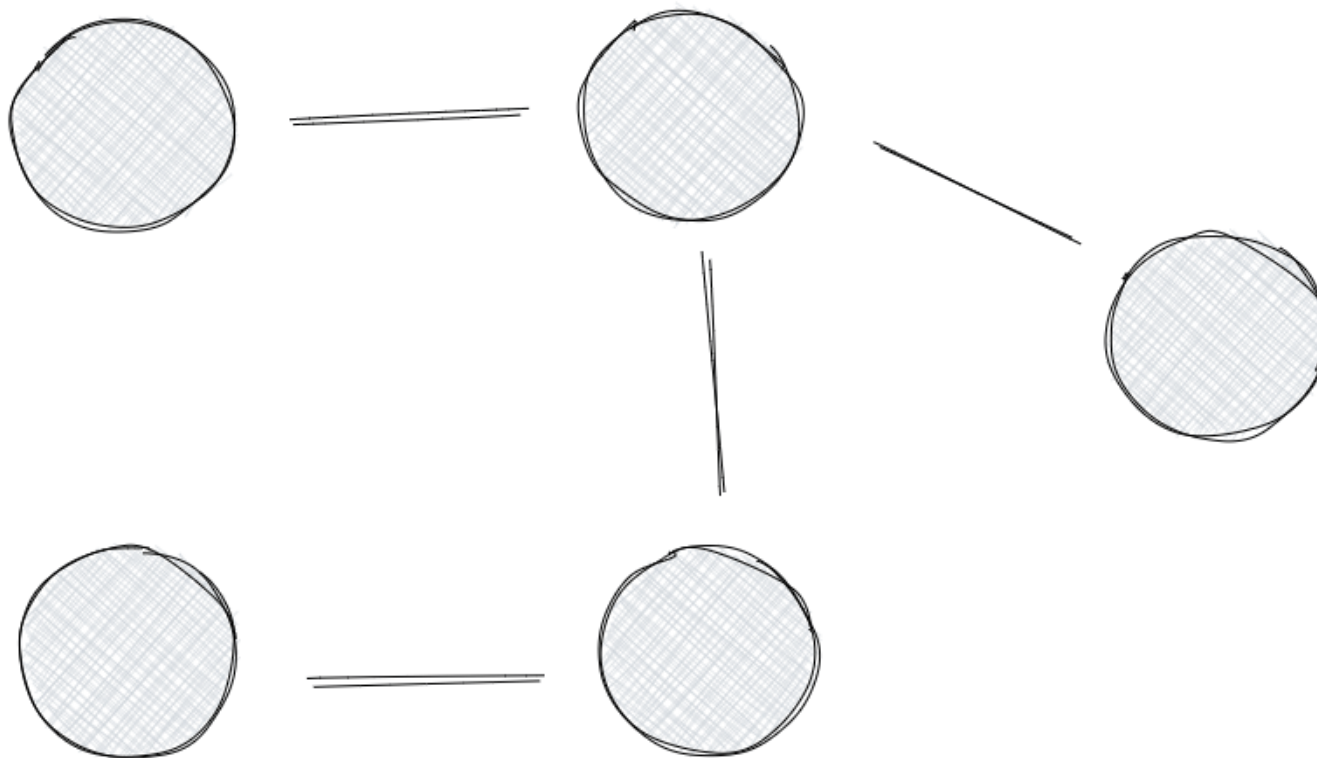
Ми чітко бачимо, що цей граф має дві зв'язані компоненти.



# Основні концепції – Зв'язність

**Дерево** — це зв'язний граф, який складається з  $n$  вузлів і  $n-1$  ребер. Між будь-якими двома вузлами дерева існує унікальний шлях. Ви також можете думати про дерево як про зв'язний граф, який не містить жодних циклів.

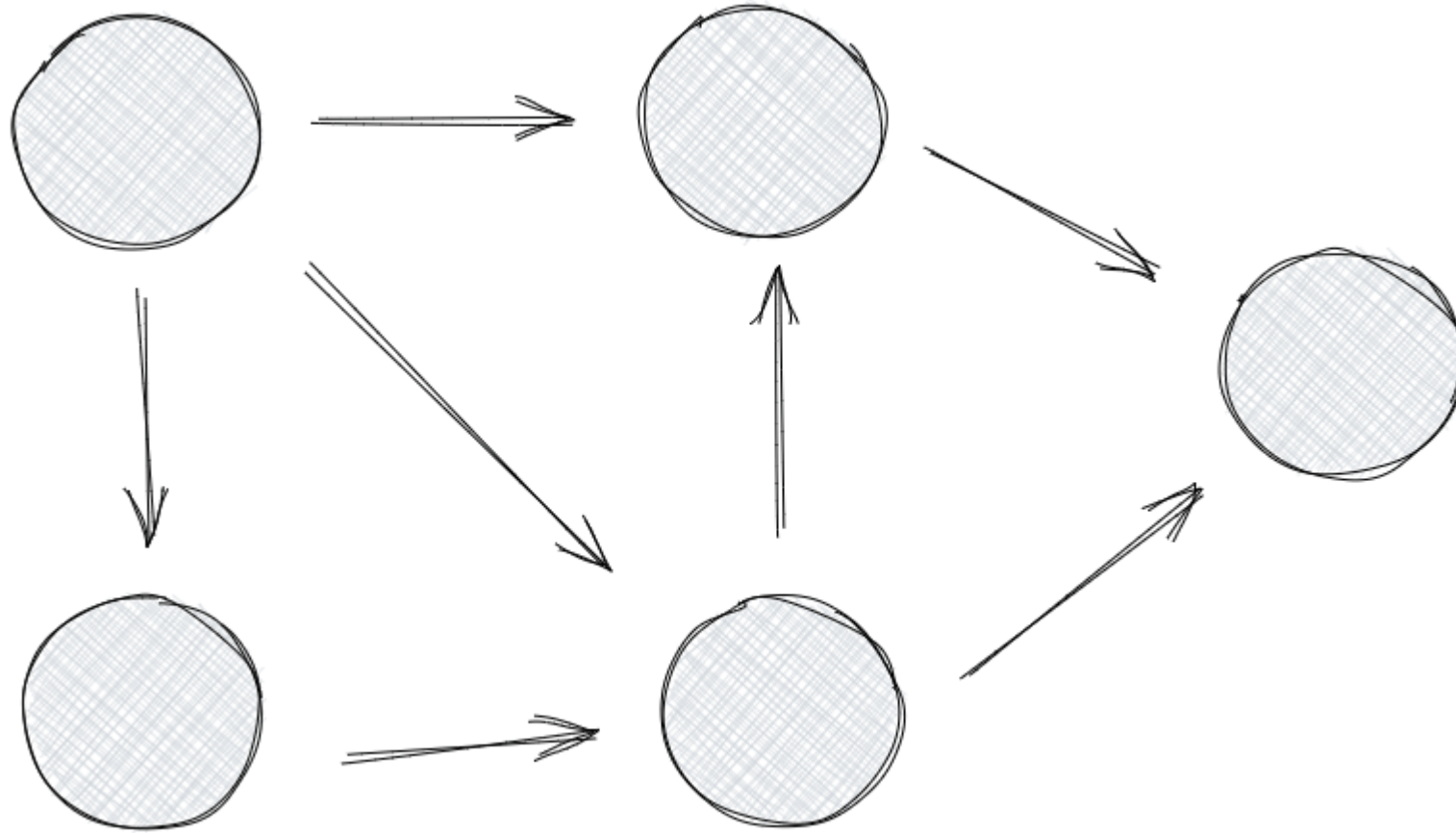
Наступний граф є деревом:



*Спробуйте самі: візьміть будь-яку пару вузлів у попередньому графі та перевірте, що між ними дійсно існує **унікальний** шлях.*

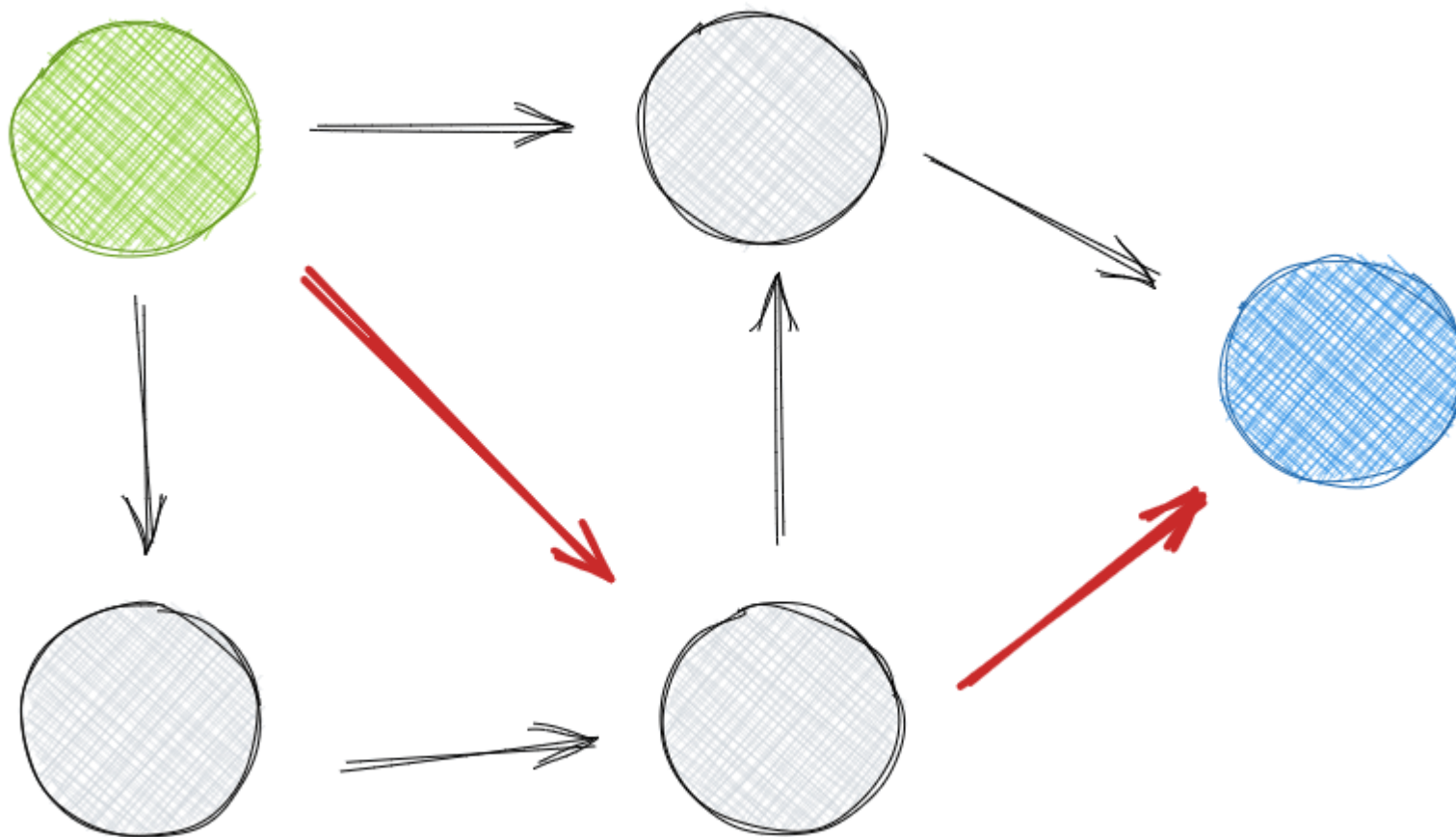
# Основні концепції – Напрямок (Орієнтація) і Ваги

Граф є **орієнтованим**, якщо ребра можуть бути пройдені лише в одному напрямку.  
Наприклад, наступний граф є орієнтованим:



# Основні концепції – Напрямок (Орієнтація) і Ваги

Зверніть увагу, як на наступному зображенні існує шлях (насправді більше одного) між зеленим вузлом і синім вузлом, але зворотне не є вірним. Ви не можете знайти жодного шляху, що починається в синьому вузлі і закінчується в зеленому:



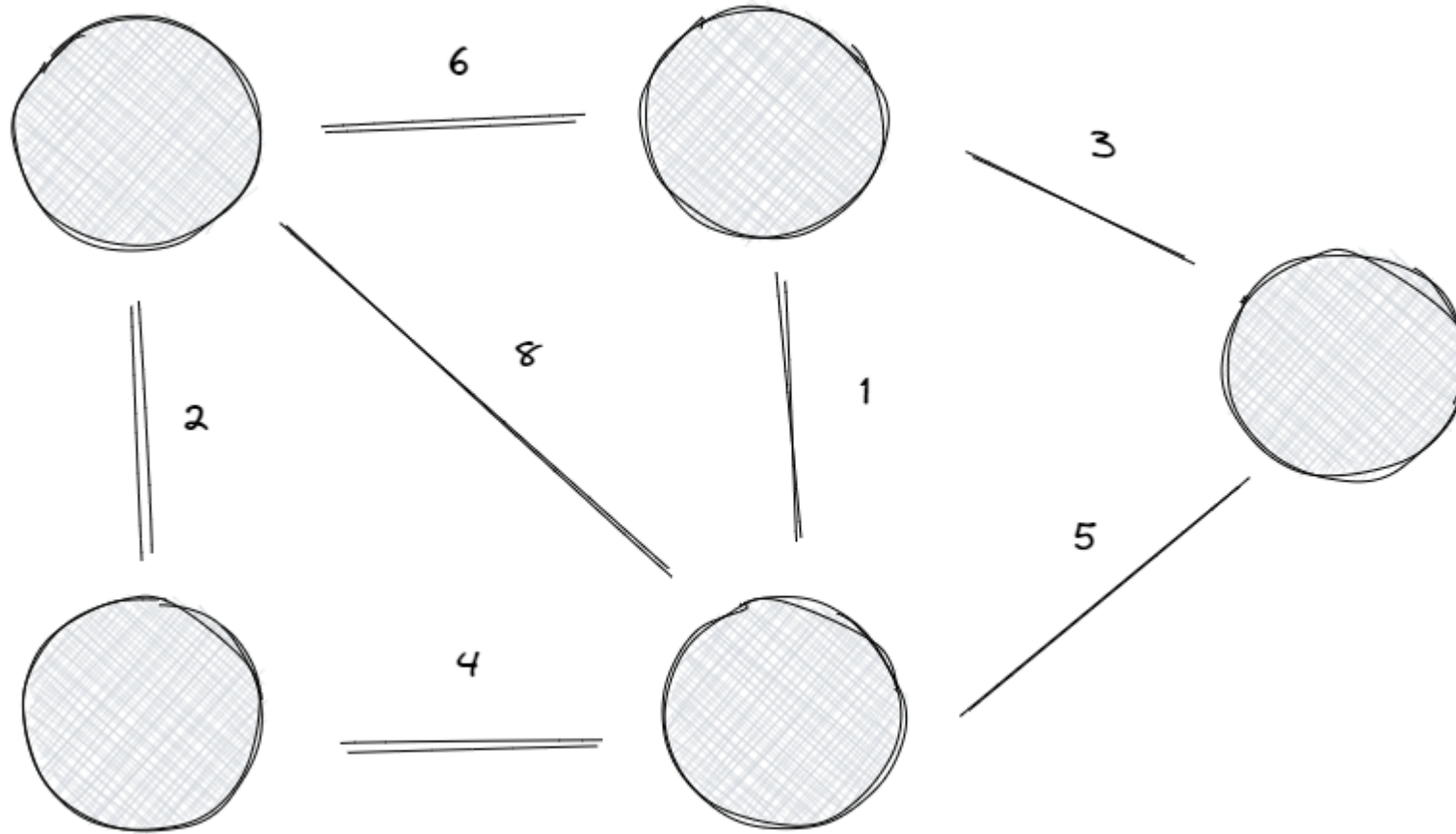
Як ми бачимо, факт, що граф є орієнтованим, впливає на концепцію зв'язності, яка називається **сильною зв'язністю**, коли ми говоримо про орієнтовані графи.

# Основні концепції – Напрямок (Орієнтація) і Ваги

У зваженому графі кожному ребру присвоюється **вага**.

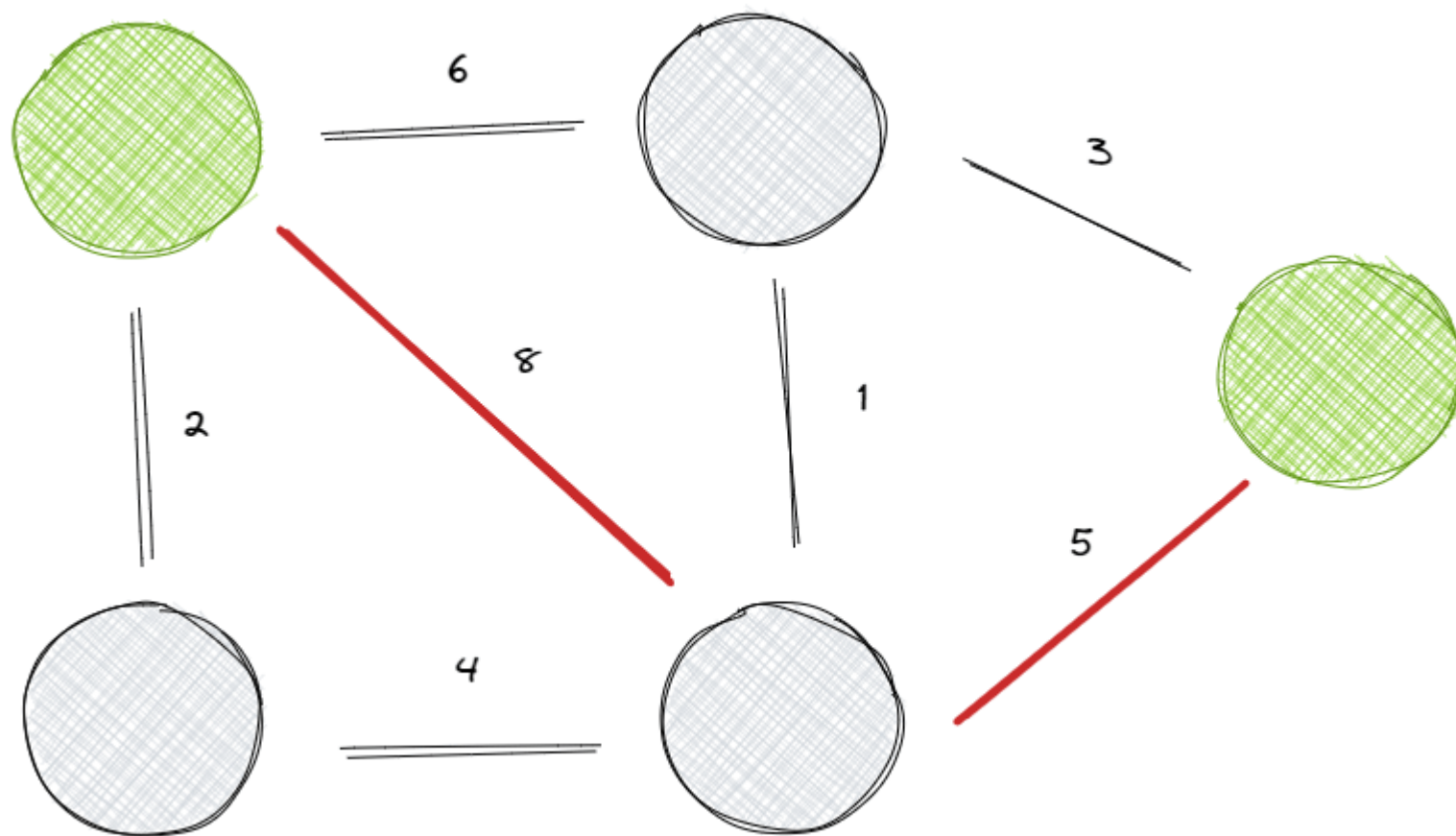
**Ваги** часто інтерпретуються як довжини ребер.

Наступний граф є зваженим:

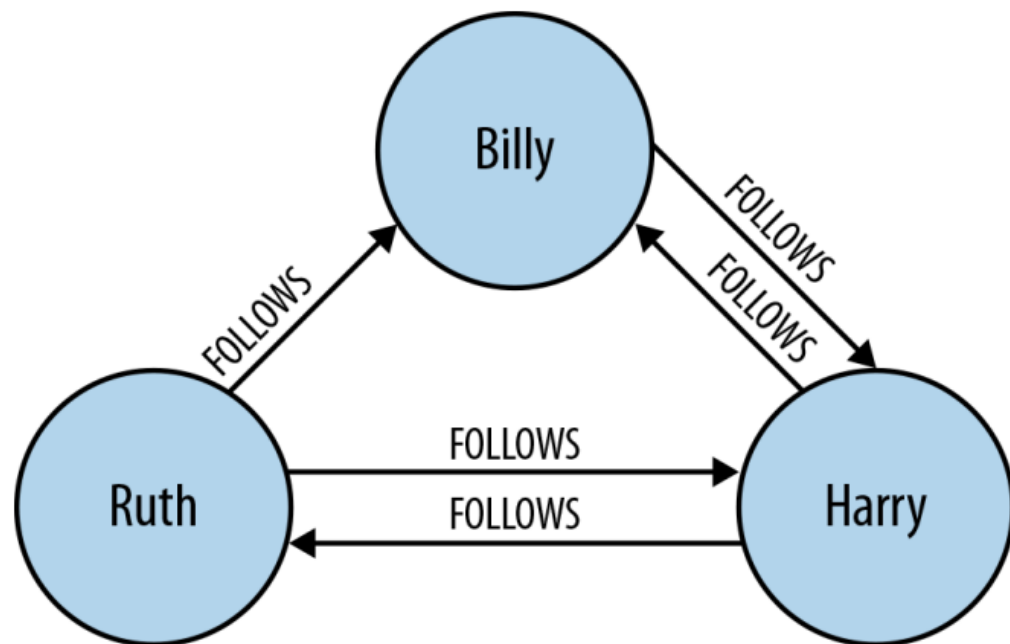


# Основні концепції – Напрямок (Орієнтація) і Ваги

Довжина шляху в зваженому графі — це **сума** ваг ребер на шляху. Наприклад, у наступному графі довжина виділеного шляху між зеленими вузлами становить 13 ( $8 + 5$ ):



# Приклад - Невеликий соціальний граф



Дані у Twitter легко уявити у вигляді графа.

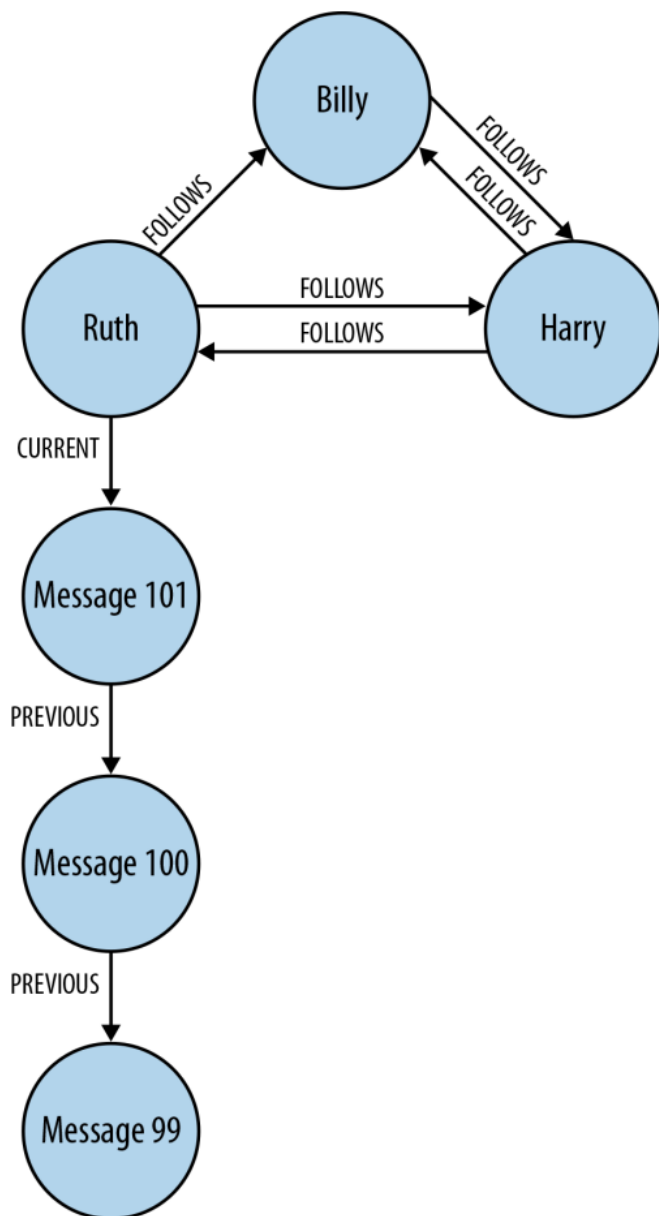
Ми бачимо невелику мережу підписників.

Взаємовідносини тут є ключовими для встановлення семантичного контексту:

- Біллі підписаний на Гаррі, і що Гаррі, у свою чергу, підписаний на Біллі.
- Рут і Гаррі також підписані один на одного, але, на жаль, хоча Рут підписана на Біллі, Біллі (поки що) не відповів взаємністю.

Звісно, реальний граф Twitter у сотні мільйонів разів більший за приклад, але він працює за тими ж самими принципами.

# Приклад - Невеликий соціальний граф



Ми розширили граф, включивши до нього повідомлення, опубліковані Рут.

Незважаючи на свою простоту, приклад демонструє виразну потужність графового моделювання.

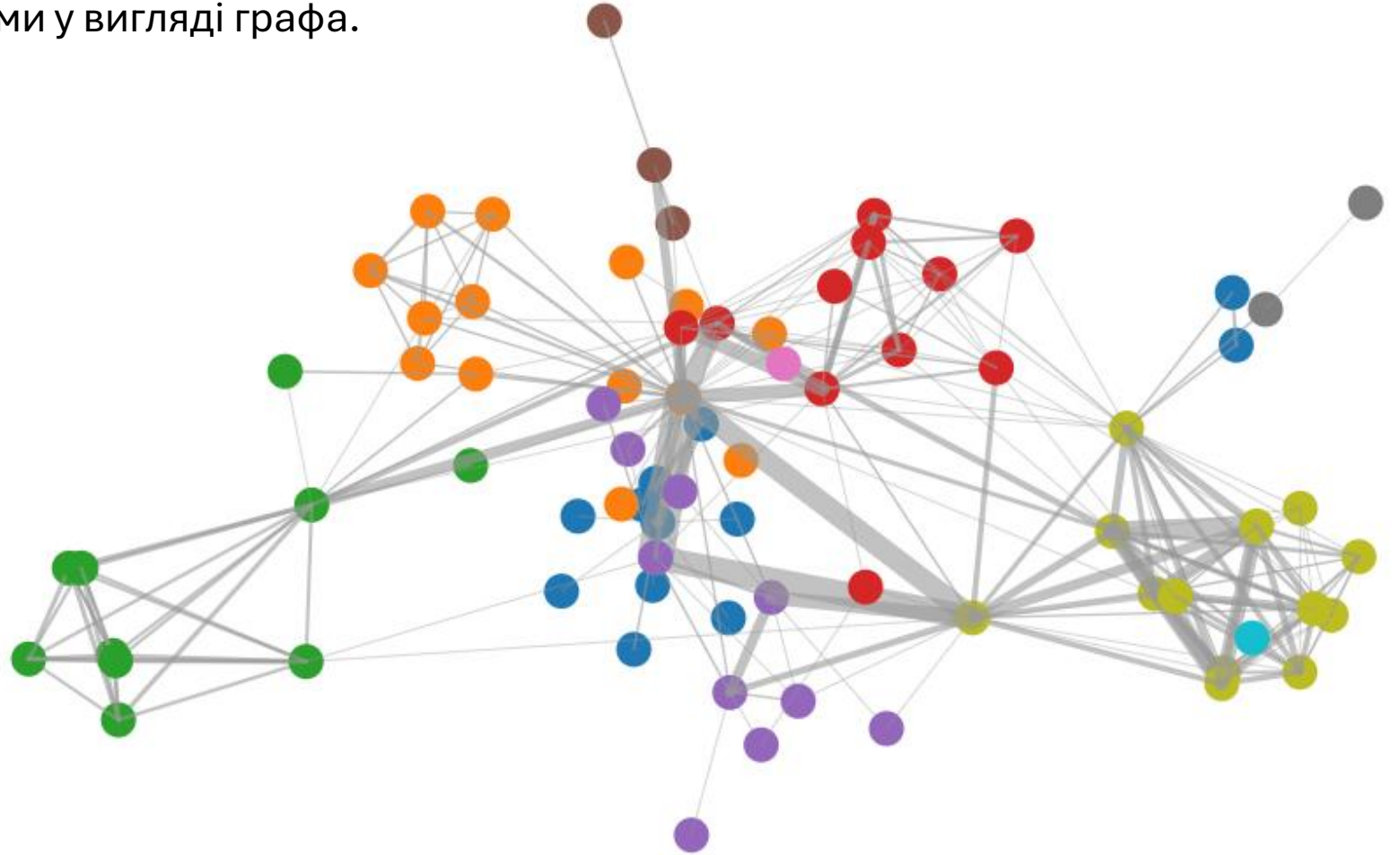
З нього легко зрозуміти, що Рут опублікувала послідовність повідомлень.

Найновіше повідомлення можна знайти, слідуючи за взаємозв'язком, позначеним як CURRENT; Далі ідуть PREVIOUS, що створюють часову лінію постів.

# Графові мови запитів

# Мови запитів

**Мова запитів до графів** — це спеціалізована мова програмування для взаємодії з графовими базами даних. Вона дозволяє користувачам отримувати, оновлювати та маніпулювати даними, змодельованими у вигляді графа.



# Мови запитів

Існує кілька мов запитів, спеціально розроблених для роботи з графовими базами даних. Кожна з них має свої особливості та підходить для різних типів графових баз даних. Вибір конкретної мови запитів залежить від вашої графової бази даних та вимог до запитів і аналізу даних.

| Деякі з найпоширеніших мов запитів   |                                                                                                                                                                |                                                                                         |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Мова запитів                         | Опис                                                                                                                                                           | Особливості                                                                             |
| Cypher                               | Мова запитів для <b>Neo4j</b> , яка дозволяє легко писати запити для графових даних.                                                                           | Інтуїтивний синтаксис, схожий на SQL, підтримка складних запитів і графових алгоритмів. |
| Gremlin                              | Мова запитів для <b>Apache TinkerPop</b> , яка підтримується багатьма графовими базами даних, такими як <b>JanusGraph</b> , <b>Amazon Neptune</b> та інші.     | Потужний і гнучкий синтаксис, підтримка складних графових операцій.                     |
| SPARQL                               | Мова запитів для <b>RDF</b> графів, розроблена консорціумом <b>W3C</b> , яка використовується в семантичних веб-технологіях.                                   | Підтримка семантичного пошуку, робота з трійками (суб'єкт-предикат-об'єкт).             |
| GraphQL                              | API query language для веб-сервісів, розроблена <b>Facebook</b> , <u>НЕ є мовою запитів до графових БД</u> , але може використовуватись як API layer над ними. | Гнучкість у запитах до API, можливість отримувати тільки необхідні дані.                |
| AQL (ArangoDB Query Language)        | Мова запитів для <b>ArangoDB</b> , яка підтримує графи, документи та ключ-значення.                                                                            | Потужний синтаксис, підтримка складних запитів і графових операцій.                     |
| PGQL (Property Graph Query Language) | Мова запитів для <b>Property Graph</b> , розроблена <b>Oracle</b> .                                                                                            | Схожий на SQL синтаксис, підтримка складних графових запитів.                           |

# Мови запитів

| Мова запитів                             | Приклад                                                                                                                                                                                                 |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQL (реляційна база даних)               | <pre>SELECT u2.name FROM Users u1 JOIN Friends f ON u1.id = f.user_id JOIN Users u2 ON f.friend_id = u2.id WHERE u1.name = 'Alice';</pre>                                                               |
| Cypher                                   | <pre>MATCH (alice:Person {name: 'Alice'})-[:FRIEND]-&gt;(friend) RETURN friend.name</pre>                                                                                                               |
| Gremlin                                  | <pre>g.V().has('name', 'Alice').out('FRIEND').values('name')</pre>                                                                                                                                      |
| SPARQL                                   | <pre>SELECT ?friendName WHERE {   ?alice &lt;http://example.org/name&gt; "Alice" .   ?alice &lt;http://example.org/knowns&gt; ?friend .   ?friend &lt;http://example.org/name&gt; ?friendName . }</pre> |
| GraphQL                                  | <pre>{ person(name: "Alice") { friends { name } } }</pre>                                                                                                                                               |
| AQL (ArangoDB Query Language)            | <pre>FOR v, e, p IN 1..1 OUTBOUND 'persons/Alice' GRAPH 'social' RETURN v.name</pre>                                                                                                                    |
| PGQL 2.0 (Property Graph Query Language) | <pre>SELECT friend.name MATCH (alice:Person)-[:FRIEND]-&gt;(friend) WHERE alice.name = 'Alice'</pre>                                                                                                    |

# Як вибрати мову запитів до графів?

Вибір правильної мови запитів до графів може значно вплинути на ефективність і продуктивність ваших операцій з даними. Щоб зробити обґрунтований вибір, врахуйте наступні ключові аспекти:

| Ключовий аспект                  | Опис                                                                                                                                                                                                                   |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Сумісність з моделлю даних       | Мова запитів повинна відповідати структурі ваших даних. Наприклад, для Property Graphs підходять <b>Cypher</b> або <b>Gremlin</b> , а RDF графів — <b>SPARQL</b> .                                                     |
| Сумісність з базою даних         | Переконайтеся, що мова запитів сумісна з вашою графовою базою даних. Деякі мови, як <b>Gremlin</b> , є універсальними, тоді як інші тісно пов'язані з конкретними базами даних.                                        |
| Складність запитів               | Оцініть складність ваших запитів. <b>Cypher</b> підходить для більшості запитів завдяки декларативному синтаксису, тоді як <b>Gremlin</b> забезпечує розширені можливості для складних графових обходів та алгоритмів. |
| Екосистема та спільнота          | Активна спільнота забезпечує кращу документацію, інструменти та підтримку, що може значно допомогти в розробці та усуненні неполадок.                                                                                  |
| Продуктивність і масштабованість | Проаналізуйте, наскільки добре мова запитів справляється з великими наборами даних і чи може вона ефективно оптимізувати ваші запити.                                                                                  |
| Крива навчання                   | Оцініть, наскільки швидко Ви та Ваша команда зможете освоїти мову запитів.                                                                                                                                             |
| Специфіка випадку використання   | Враховуйте конкретний випадок використання вашого проекту. Наприклад, <b>GraphQL</b> , як API layer, підходить для фронтенд-розробки, а <b>Cypher</b> — для запитів до графових баз даних.                             |

# Переваги та Недоліки



# Переваги графових баз даних

**Продуктивність:** Графові бази даних забезпечують значний приріст продуктивності під час роботи зі взаємозв'язаними даними порівняно з реляційними базами даних та NoSQL-сховищами.

**Гнучкість:** Графові бази даних дозволяють легко організовувати взаємозв'язки між даними відповідно до змінних потреб, додаючи нові види взаємозв'язків, вузли, мітки та підграфи без порушення існуючих запитів та функціоналу.

**Оперативність:** Сучасні графові бази даних підтримують ітераційні методи розгортання ПЗ, мають вбудовану графову модель даних, API та мову запитів, що дозволяє ефективно розробляти та обслуговувати додатки.



# Переваги графових баз даних

**Масштабованість:** Графові бази даних легко масштабуються як вертикально, так і горизонтально, що дозволяє ефективно обробляти великі обсяги даних та складні запити.

**Природність моделювання:** Графові бази даних природно відображають реальні взаємозв'язки між об'єктами, що робить їх ідеальними для моделювання соціальних мереж, рекомендаційних систем, управління ланцюгами постачання тощо.

**Швидкість запитів:** Завдяки оптимізованій структурі даних, графові бази даних забезпечують швидке виконання запитів, особливо для складних запитів, що включають багато взаємозв'язків.

**Інтуїтивність:** Графові моделі даних є інтуїтивно зрозумілими для розробників, що спрощує процес проектування та розробки додатків.



# Переваги графових баз даних

**Аналіз даних:** Графові бази даних дозволяють легко виконувати складний аналіз даних, включаючи пошук шляхів, виявлення шаблонів та аналіз впливу.

**Безпека:** Багато графових баз даних підтримують детальне управління доступом, що дозволяє забезпечити високий рівень безпеки даних.

**Інтеграція:** Графові бази даних легко інтегруються з іншими системами та технологіями, що робить їх гнучким інструментом для різних бізнес-завдань.



# Виклики та обмеження

**Складність впровадження та підтримки:** Вимагають спеціалізованих знань через складність зв'язків.

**Відсутність стандартів:** Існує багато різних графових баз даних і мов запитів (наприклад, Cypher, Gremlin), що може ускладнити вибір, перехід між системами та інтеграцію.

**Менш зріла екосистема:** Графові бази даних є відносно новою технологією. Це може означати меншу кількість доступних інструментів, бібліотек і фахівців.

**Висока вартість:** Вищі витрати на впровадження та підтримку через складнішу архітектуру, вищі вимоги до обробки, навчання персоналу та інтеграцію з існуючими системами.

**Обмежена масштабованість для певних завдань:** Можуть мати проблеми з масштабованістю для великих обсягів даних.

**Обмежена підтримка транзакцій:** Хоча деякі графові бази даних підтримують транзакції, їхня реалізація може бути менш потужною або менш ефективною порівняно з реляційними базами даних.



# Рейтинг графових баз даних

| Rank     |          |          | DBMS                      | Database Model | Score    |          |          |
|----------|----------|----------|---------------------------|----------------|----------|----------|----------|
| Nov 2025 | Oct 2025 | Nov 2024 |                           |                | Nov 2025 | Oct 2025 | Nov 2024 |
| 1.       | 1.       | 1.       | Neo4j                     | Graph          | 52.36    | -0.15    | +9.66    |
| 2.       | 2.       | 2.       | Microsoft Azure Cosmos DB | Multi-model    | 22.55    | -0.16    | -1.40    |
| 3.       | 3.       | 3.       | Aerospike                 | Multi-model    | 4.29     | -0.24    | -1.04    |
| 4.       | 4.       | 5.       | ArangoDB                  | Multi-model    | 2.97     | +0.06    | -0.12    |
| 5.       | 5.       | 4.       | Virtuoso                  | Multi-model    | 2.91     | +0.17    | -0.96    |
| 6.       | 6.       | 6.       | OrientDB                  | Multi-model    | 2.59     | -0.03    | -0.39    |
| 7.       | 8.       | 7.       | GraphDB                   | Multi-model    | 2.38     | +0.07    | -0.52    |
| 8.       | 7.       | 10.      | NebulaGraph               | Graph          | 2.37     | -0.08    | +0.58    |
| 9.       | 9.       | 9.       | Amazon Neptune            | Multi-model    | 2.36     | +0.09    | +0.19    |
| 10.      | 10.      | 8.       | Memgraph                  | Graph          | 2.17     | -0.08    | -0.55    |
| 11.      | 11.      | 12.      | JanusGraph                | Graph          | 1.81     | +0.01    | +0.03    |
| 12.      | 12.      | 14.      | TigerGraph                | Graph          | 1.73     | -0.04    | +0.32    |
| 13.      | 13.      | 11.      | Stardog                   | Multi-model    | 1.72     | -0.02    | -0.07    |
| 14.      | 14.      | 13.      | Fauna                     | Multi-model    | 1.47     | -0.02    | +0.04    |
| 15.      | 16.      | 17.      | SurrealDB                 | Multi-model    | 1.37     | +0.10    | +0.36    |
| 16.      | 15.      | 15.      | Dgraph                    | Graph          | 1.28     | -0.03    | -0.05    |
| 17.      | 17.      | 16.      | Giraph                    | Graph          | 1.07     | +0.00    | -0.03    |
| 18.      | 18.      | 18.      | Blazegraph                | Multi-model    | 0.76     | -0.01    | +0.01    |
| 19.      | 19.      | 19.      | AllegroGraph              | Multi-model    | 0.73     | -0.01    | -0.02    |
| 20.      | 20.      | 20.      | TypeDB                    | Multi-model    | 0.63     | -0.02    | 0.00     |
| 21.      | 21.      | 21.      | Graph Engine              | Multi-model    | 0.53     | +0.00    | -0.06    |
| 22.      | 22.      | 22.      | Fluree                    | Graph          | 0.33     | 0.00     | -0.02    |
| 23.      | 24.      | 23.      | InfiniteGraph             | Graph          | 0.27     | 0.00     | -0.02    |
| 24.      | 23.      | 24.      | RDFox                     | Multi-model    | 0.24     | -0.03    | +0.00    |
| 25.      | 25.      | 33.      | FalkorDB                  | Graph          | 0.23     | -0.03    | +0.13    |
| 26.      | 26.      | 26.      | AnzoGraph DB              | Multi-model    | 0.23     | -0.01    | +0.00    |
| 27.      | 28.      | 37.      | ArcadeDB                  | Multi-model    | 0.17     | 0.00     | +0.14    |
| 28.      | 29.      | 25.      | Apache HugeGraph          | Graph          | 0.14     | -0.02    | -0.10    |
| 29.      | 32.      | 27.      | FlockDB                   | Graph          | 0.12     | +0.00    | -0.06    |
| 30.      | 27.      | 31.      | Ultipa                    | Graph          | 0.12     | -0.06    | +0.01    |
| 31.      | 29.      | 28.      | AeneasGraph               | Multi-model    | 0.12     | 0.00     | -0.06    |

## Популярні графові бази даних

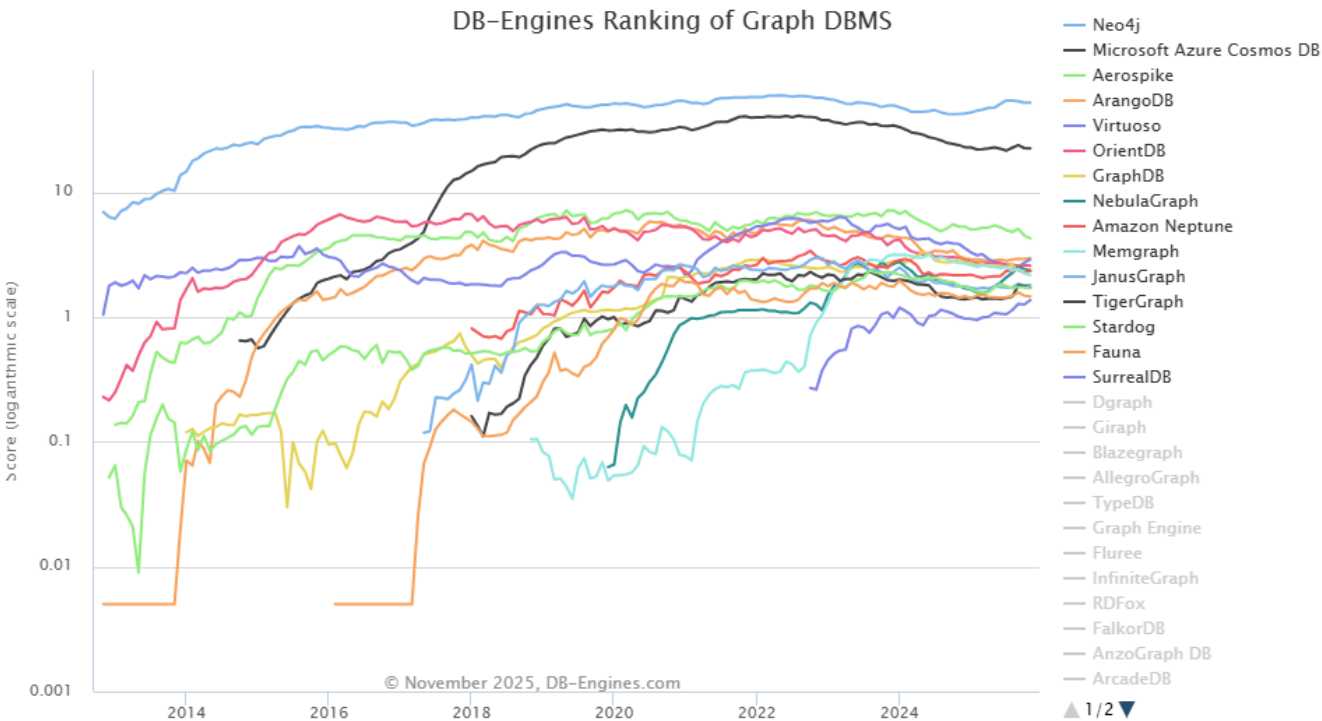
**Neo4j:** Одна з найпопулярніших графових баз даних, яка використовує мову запитів Cypher.

**Cosmos DB:** Глобально розподілена база даних від Microsoft, яка підтримує кілька моделей даних, включаючи графи, і використовує мову запитів Gremlin.

**ArangoDB:** Мульти-модельна база даних, яка підтримує графи, документи та ключ-значення.

**OrientDB:** Підтримує як графові, так і документні моделі даних.

**Amazon Neptune:** Керована графова база даних від Amazon Web Services, яка підтримує як Property Graph, так і RDF моделі.



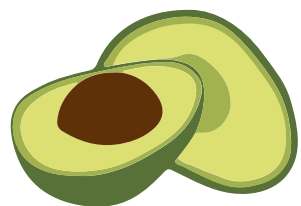
# Приклади графових баз даних



Amazon Neptune



Azure Cosmos DB



ArangoDB



**ДЯКУЮ ЗА УВАГУ!**